

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



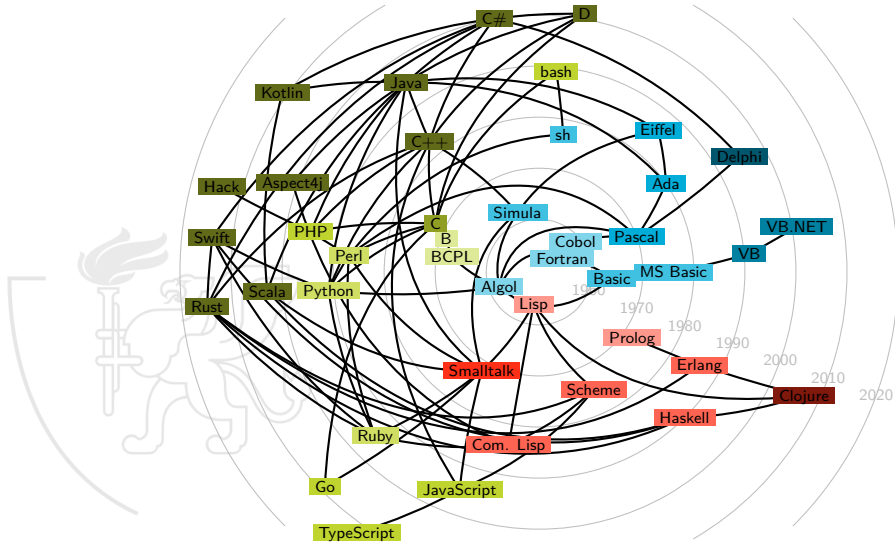
Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek, fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Példák (1. verzió)
 - Példák (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listák adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 -
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

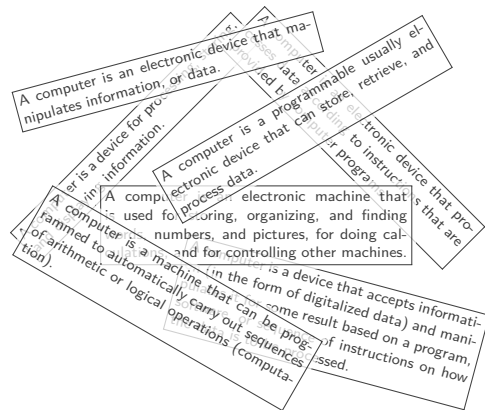
Programozási nyelvek/paradigmák



Programozás

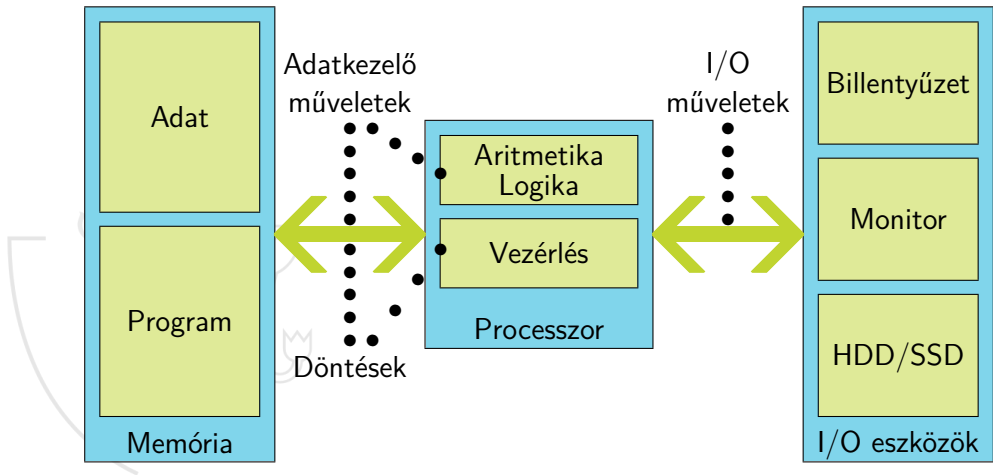
Valós problémák számítógépes megoldása.

- Számítógép tulajdonságai
 - Általános célú (univerzális)
 - A határok egyre inkább elmosódnak (pl. mobil, set-top-box, digitális tv, okos eszközök).
 - Automatikus vezérlésű
 - A programját külső beavatkozás nélkül képes végrehajtani.
 - Elektronikus
 - Az áramköri kapcsolat nem mechanikus.
 - Digitális
 - Két értékből (0,1) felépített jól elkülöníthető állapotok.
 - ...leglábbis még többségében az.



A számítógép modellje

Egyszerűsített Neumann-architektúra



A számítógép modellje

Nyelv

- A számolási tevékenységet végző ember számára egy (általa ismert) természetes nyelven lehet olyan utasításokat adni, amit képes végrehajtani.
- A számítógép számára a vezérlő egység által értelmezhető és végrehajtható utasítások (parancsok) adhatók.
- Kezdetben az ember megtanulta ezeket az utasításokat (a számítógép nyelvét). A kommunikáció ezen a nyelven lassú, nehézkes, sok hibával jár.
- Miért nem tanulja meg inkább a számítógép az emberi nyelvet?
 - Kezdi megtanulni (pl. ChatGPT), de
 - egyrészt nem véletlenül vannak még programozók,
 - másrészt a természetes nyelv nem a legalkalmasabb az egyértelmű kommunikációra.
- Megoldás: *Az ember is és a számítógép is tanuljon meg egy nyelvet!*

A számítógép modellje

Algoritmus

- A nyelv egy kifejező eszköz valaminek a leírására.
- De mit akarunk a számítógép számára leírni?
- Azt, hogy mit csináljon, vagyis milyen tevékenységet hajtson végre.
- Egy számítógép csak azt tudja megcsinálni, amit megmondunk neki, de amiről meg tudjuk mondani, hogy hogyan kell csinálni, azt a számítógép meg tudja csinálni!
- Alapvetően azt kell megmondani a számítógépnek, hogy milyen adatokon milyen műveleteket kell elvégezni.
- Ezt pontosan meg kell fogalmazni, vagyis el kell készíteni a problémát megoldó algoritmust.

Algoritmus

Adott típusú összes feladat megoldására vonatkozó pontos előírás, amely megmondja, hogy a kezdeti adatokon milyen műveleteket milyen sorrendben kell elvégezni.

- Az algoritmus tulajdonságai
 - Meghatározott: Véges módon leírható, ugyanazon bemeneti adatokra mindig ugyanazt eredményezi.
 - Széleskörű: Egész feladatosztályra vonatkozik.
 - Véges: Véges számú lépésben véget ér.
 - Nem árt, ha potenciálisan megvalósítható.
- Érdekességek:
 - Nincs olyan algoritmus, amelyik megmondja, hogy egy tetszőleges feladatra hogyan kell algoritmust készíteni.
 - Nincs olyan algoritmus, amelyik el tudja dönteni, hogy egy tetszőleges program a kezdőadatokkal végrehajtva véges lépés után megáll-e.

Programozási nyelv

Olyan nyelv (szintaktikai és szemantikai szabályok összessége), amely egy problémát megoldó algoritmus leírására szolgál.

- Programozási nyelv lehet ...
 - ... az emberi nyelvhez közel álló (magas szintű)
 - ... a számítógép nyelvéhez közel álló (alacsony szintű)

Program

Egy algoritmusnak egy adott programozási nyelven történő leírása.

Hardver

A számítást végző fizikai-technikai rendszer.

Szoftver

A hardvert működtető programok és parancsok összessége.

Adat

A hardver és a szoftver által feldolgozott információ.

- A kommunikáció ember és gép között véges jelhalmazból (ábécé) vett, meghatározott formai szabályokat kielégítő, véges hosszúságú jelsorozatokkal történik. Ezek a jelsorozatok alkotják a kommunikáció nyelvét.
- A nyelvet a szintaxis és a szemantika együtt határozza meg.

Szintaxis

Formai szabályok olyan rendszerét, amely meghatározza, hogy egy adott kommunikációs nyelvben melyek a szabályos jelsorozatok, a nyelv szintaxisának nevezzük.

Szemantika

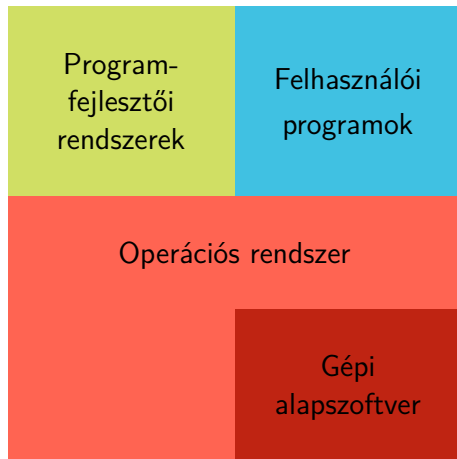
Egy nyelv szemantikája határozza meg, hogy a szintaktikusan helyes jelsorozatok mit jelentenek.

- A szintaxis megadására számos módszer ismeretes (szintaxis diagram, reguláris kifejezés, generatív nyelvtan, ...)
- Mi a Python nyelv szintaxisát példákkal és magyarázatokkal fogjuk megadni.
- Példák:
 - **Számjegy:** Egy tízes számrendszerbeli számjegy karakter (`[0-9]`), pl.: 4, 2
 - **Betű:** Az angol ábécé egy kis vagy nagybetűje (`[A-Za-z]`), pl.: N, k
 - **Python azonosító:** Betűvel vagy `'_'` karakterrel kezdődő, betűkből, számjegyekből és `'_'` karakterből álló nemüres jelsorozat (`[_A-Za-z][_0-9A-Za-z]*`), pl: `__name__`, `alfa`, `WELCOME_STR`, `CamelCase`, `everything42`



- Nagyon sokféle hardver létezik, de az alapfelépítése mindnek nagyon hasonló.
 - Központi feldolgozó egység (CPU, Processzor)
 - Jellemzők: felépítés (CISC, RISC), sebesség (MHz, GHz), magok száma
 - Memória
 - Jellemzők: típus (ROM, RAM), méret (bit, byte, kB, KiB, MB, MiB, GB, GiB)
 - Háttértár (HDD, SSD, CD, DVD, BD, Flash)
 - Jellemzők: méret (MB, MiB, GB, GiB, TB, TiB)
 - Felhasználói terminál és perifériák
 - Billentyűzet, egér, képernyő, hanggenerátor, nyomtató, modem, hálózati csatoló, speciális eszközök

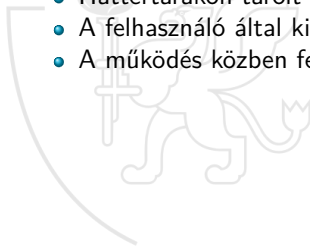
- Egy hierarchikus felépítésű programrendszer.
- Főbb szintjei felhasználó szemszögéből:
 - Felhasználói programok
 - Programfejlesztői rendszerek
 - Operációs rendszer
 - Gépi alapszoftver
- Minden szint elemei előállíthatók programfejlesztői rendszerek segítségével



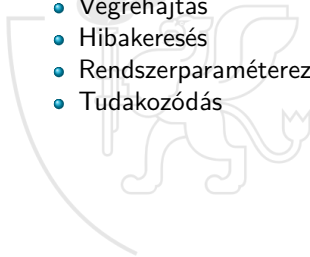
- A hardver alapvető működését biztosító, általában ROM-ba (EPROM, EEPROM) égetett szoftver (Basic Input Output System).
- Főbb funkciói:
 - A hardver alapvető konfigurációja
 - A hardver tesztelése
 - Az operációs rendszer betöltése és indítása
 - Gépi szintű be- és kimenet megvalósítása (ezt a feladatot a modern operációs rendszerek az elindításuk után részben vagy teljesen átveszik)



- Olyan programrendszer, amely közvetítő szerepet tölt be a számítógép hardver erőforrásai és a felhasználó között.
- Főbb funkciói:
 - Programok betöltése és végrehajtása
 - Erőforrások elosztása
 - Input/output műveletek végzése
 - Háttértárakon tárolt adatrendszerek kezelése
 - A felhasználó által kiadott parancsok értelmezése és végrehajtása
 - A működés közben fellépett hibák lekezelése



- Feladata, hogy támogassa a programozás során a programok (programrendszerek) létrehozását, módosítását és végrehajtását.
- Főbb funkciói:
 - Könyvtárkezelés
 - Szövegszerkesztés
 - Fordítás
 - Végrehajtás
 - Hibakeresés
 - Rendszerparaméterezés
 - Tudakozódás



- A megoldandó probléma számítógépes megoldását szolgáltatja.



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek, fogalmak**
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Programozási nyelv

Olyan nyelv (szintaktikai és szemantikai szabályok összessége), amely egy problémát megoldó algoritmus leírására szolgál.

- Programozási nyelv lehet ...
 - ... az emberi nyelvhez közel álló (magas szintű)
 - ... a számítógép nyelvéhez közel álló (alacsony szintű)
- Azonosító
- Adattípus
- Deklaráció / Definíció
- Változó / Konstans
- Függvény / Metódus
- Literál / Kifejezés

Azonosító

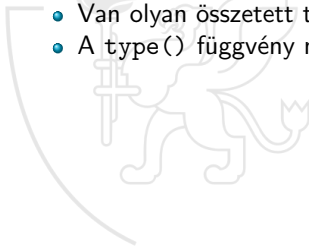
(Programozási nyelvekben) egy név, mely a különféle programelemek (egyértelmű) azonosítására szolgál

- Egy programozási nyelvben az azonosító egy szintaktikai elem, egy adott formai szabályok alapján képzett jelsorozat
- Python-ban:
 - Betűvel vagy '_' karakterrel kezdődő, betűkből, számjegyekből és '_' karakterből álló nemüres jelsorozat (`[_A-Za-z][_0-9A-Za-z]*`), pl: `__name__`, `alfa`, `WELCOME_STR`, `CamelCase`, `everything42`

Adattípus

Az adattípus a programnak egy olyan komponense, amely két összetevője, az értékhalmaz és az értékhalmaz elemein végezhető műveletek által meghatározott.

- Minden adattípus vagy elemi, vagy más adattípusokból képzett összetett adattípus.
- Python-ban:
 - Van olyan összetett típus, amely Python-ban nem létező típusból képződik.
 - A `type()` függvény megadja egy érték típusát.



Deklaráció

Egy programkomponens deklarációja egy azonosító (név) hozzárendelése az adott komponenshez.

- Ezzel az azonosítóval hivatkozhatunk a program további részében az adott komponensre (adatra, műveletre, értékre, adattípusra).
- A program egy adott pontján csak azok a komponensek használhatók (hivatkozhatók), amelyeket e pontot megelőzően már deklaráltunk, ellenkező esetben hiba lép fel.
- Python-ban:
 - Nincs deklaráció, más nyelveken létezik.
 - Egy azonosítónak az első (végrehajtott) definíciója egyben a deklarációjának is tekinthető, ezután lesz használható, addig nem létezik. (Ha mégis használni próbáljuk, *futási hibát* kapunk.)
 - Python-ban lehetőség van az azonosítók *törlésére* (`del`), vagyis a deklaráció „visszavonására”.

Definíció

Egy programkomponens definíciója egy „érték” hozzárendelése a komponens azonosítójához.

- Ebben a kontextusban az „érték” eléggé tág fogalom, jelenthet például (függvények esetében) utasítássorozatot, algoritmust is.
- Egy adott azonosító lehet egyszerre *deklarált* de *nem definiált*.
- Python-ban:
 - Egy azonosító az első (végrehajtott) definíciója után lesz használható.
 - Az azonosítóknak nincs típusa, csak az általuk tárolt értékeknek.
 - Ha egy azonosító értéke `None`, az azt jelenti, hogy az azonosító *definiált*, de nincs értéke.
 - Egy adott azonosító a `del` utasítással törölhető; az utasítás után az azonosító nem definiált.

Változó

A változó olyan programegység, amely műveletek hatására tetszőleges értéket felvehet, és értékét a program végrehajtása során akárhányszor megváltoztathatjuk.

- A változókban tehát értékeket tudunk tárolni későbbi felhasználás céljából.
- Python-ban:
 - Egy változó bármilyen típusú értéket tárolhat.
 - Egy változónak az értékadó utasítás (=) segítségével adhatunk értéket.
 - Egy változó a neki történő első értékadással jön létre.

Konstans

A konstans olyan komponense a programnak, amely a definíciójában megadott értéket azonosítja, és ez az érték a program végrehajtása során nem változtatható meg. Típusa a definíciója által meghatározott adattípus.

- A konstans tehát egy konkrét érték elnevezéseként is felfogható (pl. π , e , c).
- Python-ban:
 - Nyelvi szinten nincsenek konstansok: Nem tudunk olyan azonosítót létrehozni, aminek az értékét később ne tudnánk megváltoztatni.

- Python-ban nyelvi szinten **nincsenek** konstansok!
- Python kódolási stílus / ajánlás / szokás, hogy a csupa nagybetűs változókat konstansnak tekintjük, konstansként kezeljük, pl:

```
1 G = 6.6743e-11
2 C = 299_792_458
3 MOL = 6.02214076e23
4 FAIL_STR = '\033[1;31m[FAIL]\033[0m'
5 PASS_STR = '\033[1;31m[PASS]\033[0m'
6 TILTOTT_JELSZAVAK = ['1', '12', '123', 'qwe123', 'Egy aprócska kalapocska, benne csacska macska mocska.']
```

- Ezek továbbra is „csak” változók: semmi sem akadályoz meg bennünket abban – legfeljebb a professzionalizmus, a lelkiismeret, és a Python nyelv alkotói iránti tisztelet –, hogy új értéket adjunk nekik!

Függvény

A függvény a matematikai értelemben vett függvény általánosítása (bővítése), gyakorlatilag egy (rész)algoritmus megvalósítása.

- Egy-egy függvény valamilyen bemenő adatok alapján kiszámol egy értéket, mint ahogyan azt a matematikában már megszokhattuk.
- Vannak olyan függvények, amiknek valamilyen jól definiált mellékhatása is van a visszatérési érték kiszámítása mellett.
- Adott esetben az is előfordulhat, hogy egy függvénynek csak a mellékhatása fontos, és (matematikai értelemben) nem ad vissza semmilyen értéket, pl. a `print(...)`. (Az ilyen függvényeket nevezhetjük eljárásnak.)
- Python-ban:
 - Egy függvény látszólag több értéket is visszaadhat.
 - A „nem ad vissza értéket” a `None` érték visszaadásával valósul meg.

- Python nyelven függvényeket lehet definiálni és meghívni.
 - Definíciónál meg kell adnunk, hogy a függvényünk milyen paraméterekből milyen értéket hogyan fog kiszámolni, vagyis:
 - Meg kell adni a *paraméterek* listáját, és
 - meg kell adni az algoritmust.
 - A függvényhívásnál pedig konkrét *argumentumokra* alkalmazzuk a függvényt, és a kiszámított értéket felhasználhatjuk további számolásainkhoz.



Függvény

Definíció és használat [2/2]

- Függvény definíciót a `def` kulcsszó segítségével készíthetünk:

```
def f(a, b):  
    return a + b
```

- Nagyon fontos a „szépen formázottság”, vagyis a *függvény törzsének* (az algoritmusát leíró utasításoknak) a kódban egy behúzással beljebb kell kezdődniük, mint ahol a függvény definíciója kezdődik! Ez (az *indentálás*) Python-ban nem csak egy ajánlás, hanem kötelező, a nyelv szintaktikájának szerves része!
- Függvényhíváskor a függvény nevét zárójelben a hívás argumentumai követik:

```
c = f(9, 3 * 11)
```

- Python-ban a paraméterezés többféle lehet; a legegyszerűbb sorrendi argumentumoknál a definíciónál felsorolt paraméterek sorrendben kapják meg a hívásból megadott argumentumokat.

Metódus

A metódus egy olyan függvény, ami kifejezetten valamely adattípushoz tartozik, azon adattípus értékein értelmezett.

- Egy-egy metódus az adott típus (és csak ezen típus) egy értékére lehet meghívni.
- A metódus tulajdonképpen felfogható egy olyan függvénynek, ami argumentumként megkapja azt az értéket is, amire meghívtuk; ilyen tekintetben semmiben nem különbözik egy általános függvénytől: lehet mellékhatása, visszatérhet több értékkel, vagy egygyel sem.
- A metódus és a függvény közötti különbség teljes megértéséhez olyan ismeretekre lenne szükség (az OOP – objektumorientált programozás *osztály* fogalma), amiről ezen kurzus keretein belül nem lesz szó.

- Metódusok definiálásával most nem foglalkozunk.
- A metódushívásnál az adott típus egy konkrét értékére és a megadott *argumentumokra* alkalmazzuk a metódust.
- Metódushívásnál először megadjuk a típus egy értékét, majd egy `.` (pont) operátorral és a metódus nevével jelezzük, hogy erre hívjuk meg a metódust, végül zárójelben a hívás argumentumai következnek:

```
c = "valami_szöveg".upper()  
n = "Anna,Bence,Cili,Dániel".split(',')
```

- A fenti két sor végrehajtása értelmezhető úgy mint az alábbi két függvényhívás, de azok szintaktikailag nem helyesek, mert nem függvényekről, hanem metódusokról van szó!

```
c = upper("valami_szöveg")  
n = split("Anna,Bence,Cili,Dániel", ',')
```


Literál

A literál a program forráskódjában valamilyen típusú konstans érték közvetlen, vagyis nem külön azonosítóval ellátott megadása. Típusa az értékleírás által meghatározott adattípus.

- A literál tehát nem más, mint valamely típus konkrét értékének programbéli leírása.
- Különböző literálok jelölhetik ugyanazt az értéket! ($\frac{1}{2}$, $\frac{3}{6}$, 0.5, 0.500, 50%, 0.1₂)
- Python literálok:
 - -753, 0, 42, 0x2a, 0o52, 0b101010, 299_792_458
 - 3.141592653589793, 6e23, 5.39124760e-44, 2_718_281_828.459045e-9, 1.616_255e-35
 - 1.4142135623730951j
 - 'egy', "egy", """egy""", f'x = {x:.2f}', r"LF\n", '''LF\\n'''
 - [1, u'kettő', [], True], {2,3,5,7,11}, {1:'január', 'december':31}

Kifejezés

A kifejezés egy érték olyan leírása, amely műveleteket, azonosítókat és literálokat is tartalmazhat.

- A kifejezésekben literálokat, konstansokat, változókat és műveleteket (köztük függvényhívást) is alkalmazhatunk.
- Adott típusú (rész)kifejezésekre az adott típuson értelmezett műveletek használhatók.
- Python-ban:
 - A dinamikus típusoltság miatt (az értékeknek vannak típusai, nem maguknak a változóknak és kifejezéseknek) a művelet adott értékeken való végrehajthatósága a művelet végrehajtása közben dől el.